

System Architecture and Technical Design Specification

RAG-Based Language Agnostic Chatbot for Smart Query Response in Academic Institutions

1. Executive Summary and Problem Context

Campus administrative offices are overwhelmed each semester with thousands of repetitive inquiries regarding scholarship forms, fee deadlines, exam schedules, and eligibility rules. Official guidelines are published almost exclusively in English within unstructured PDF circulars and dynamic departmental web notices. Meanwhile, a significant proportion of students communicate more naturally in Hindi, regional languages, or colloquial Hinglish. This linguistic divide leads to long physical queues, communication breakdowns, and administrative fatigue.

2. Architectural Philosophy: Why Moving Beyond Naive RAG is Essential

Most basic student chatbot projects implement a Naive RAG workflow: chunking a PDF, storing text in a vector database, and passing the top matches to an LLM with a single prompt. Academic literature and real-world testing demonstrate that Naive RAG fails in institutional environments due to three critical flaws:

- **Blind Context Trust:** The system blindly passes whatever chunks the vector search returns. If the retrieved text is irrelevant or incomplete, the generator produces ungrounded hallucinations, misleading students with fabricated dates or fee amounts.
- **Failure on Exact Identifiers:** Dense vector embeddings excel at broad conceptual matching but consistently miss exact notice numbers, dates, and form codes (such as 'Notice Acad/2026/04' or 'Form 16-A').
- **Language Drift in Multi-Turn Chats:** As established by IEEE research (EduX-RAG, 2025), prompt-based chatbots suffer an average 27% language drift rate, slipping back into English when students switch languages during a conversation.

3. The Proposed Solution: An Agentic Self-Corrective Architecture

To resolve these challenges, our project introduces an Agentic Self-Corrective RAG system orchestrated through LangGraph. Rather than treating question answering as a rigid one-way script, the system runs as a state machine equipped with self-evaluation loops. It actively grades document relevance, reformulates unclear queries, verifies factual groundedness before output, enforces response-language consistency, and gracefully hands off unresolved inquiries to human administrative staff.

Layer-by-Layer Architectural Specifications

Detailed technical breakdown from data ingestion to multi-channel presentation

Layer 1: Automated Ingestion and Multi-Modal Indexing Pipeline

The ingestion pipeline continuously monitors and processes two primary institutional data streams:

- Web Scraping Engine (BeautifulSoup / bs4): Periodically monitors the college portal's active notice boards, extracting headline announcements, publication timestamps, and administrative links.
- Document Ingestion Engine (PyMuPDF): Parses uploaded institutional PDF circulars, extracting structured text, layout metadata, tables, and page boundaries.
- Metadata-Rich Chunking: Text is split using recursive semantic chunking with a 500-token window and 100-token overlap. Each chunk is tagged with structured metadata: {circular_number, date, department, page_number, source_url} to enable verifiable clickable citations.

Layer 2: Hybrid Retrieval Engine (Dense Semantic + Sparse Lexical)

Rather than relying solely on dense vectors, our architecture employs a dual-index hybrid search pipeline:

- Dense Vector Search (BGE-M3): BAAI's BGE-M3 model maps regional queries (Hindi, Marathi, Tamil, etc.) and English circulars into a shared 1024-dimensional semantic space, capturing conceptual intent across languages.
- Sparse Lexical Search (BM25): Indexes exact keywords, numerical dates, circular IDs, and form identifiers that dense semantic models often dilute.
- Reciprocal Rank Fusion (RRF): Combines the ranked candidate lists from both retrievers using score fusion (constant k=60), ensuring documents that match both semantically and lexically are prioritized.
- Cross-Encoder Re-Ranking: The top 10 fused candidates are re-evaluated by a lightweight cross-encoder to produce the final top-3 highly relevant context passages, stripping out background noise.

Layer 3: Pre-Processing and Linguistic Router

Incoming student messages undergo immediate pre-processing before entering the retrieval graph. A dedicated language detector identifies the source language. If Romanized Hindi (Hinglish) is detected, a normalization node transliterates mixed phrases and identifies key keywords. Meanwhile, a contextual query rewriter resolves follow-up pronoun ambiguity by referencing conversational state stored in memory (e.g., transforming 'When is its last date?' into 'When is the submission deadline for the 2026 post-matric scholarship?').

The LangGraph Self-Corrective Decision Engine

Stateful graph orchestration implementing automated self-grading and correction loops

1. LangGraph State Machine Design

At the heart of the system is a stateful decision graph built with LangGraph. The graph maintains an explicit state object containing the original student query, detected language, conversational history, retrieved document chunks, generated draft, and verification scores. The workflow moves through five specialized decision nodes:

- Node 1 (Document Relevance Grader): An evaluation step checks whether the retrieved passages genuinely contain the facts needed to answer the student's question. If the chunks are graded as irrelevant, the graph branches into the Query Reformulation loop rather than proceeding to generation.
- Node 2 (Query Reformulation and Web Fallback): When initial retrieval fails, this node reframes the query keywords to capture alternate phrasings. If internal PDF chunks still yield insufficient context, it queries the live web scraper for newly published notices.
- Node 3 (Grounded Generation Engine): Generates an answer strictly constrained by the verified passages. The prompt explicitly prohibits speculation and instructs the model to attach source metadata citations to every factual assertion.
- Node 4 (Hallucination and Factuality Grader): An automated verification check compares the generated draft against the source text. If the model introduced unsourced dates, fee numbers, or rules, the draft is rejected and sent back for re-synthesis.
- Node 5 (Language Consistency Validator): Directly solves the language drift flaw identified in research literature. If a student asked in Hindi but the LLM drafted an English response, this node intercepts the draft and enforces translation into the student's chosen language before dispatch.

2. Decision Flow and Routing Rules

The table below defines the autonomous transitions within the LangGraph state machine:

Decision Point	Condition Checked	Action Taken
Retrieval Assessment	Relevance score > 0.70 and key terms present in chunks.	Proceed directly to Grounded Generation Node.
Ambiguity / Context Gap	Relevance score < 0.70 on initial retrieval.	Route to Query Reformulation; re-query vector and live scraper.
Factuality Check	Generated facts lack direct grounding in reference chunks.	Reject draft; regenerate with stricter temperature and context bounds.
Language Verification	Draft response language differs from user's detected query language.	Enforce translation to user language while preserving citation tags.
Escalation Trigger	Information absent from all campus documents after reformulation.	Halt generation; display office contact card and log query in SQL.

Solutions to Real-World Failure Modes

Engineering concrete fixes for vulnerabilities uncovered in published research

1. Solving the Hinglish and Code-Mixing Challenge (Paper 4 Finding)

The Vulnerability: As highlighted in MDPI Informatics (2026), conversational models and dense embeddings degrade sharply when users employ code-mixing (such as blending Hindi and English into Romanized Hinglish: 'Scholarship form submit karne ki last date kya hai?'). Standard tokenizers split Romanized regional words into nonsensical subwords, destroying semantic retrieval.

Our Technical Fix: We implement a two-step normalization pipeline: a phonetic transliteration module identifies Indian linguistic tokens, and an intent-keyword mapping layer extracts standard query anchors before calling the vector store. This ensures seamless retrieval even when students use informal campus slang.

2. Eliminating Cross-Lingual Language Drift (Paper 2 Finding)

The Vulnerability: IEEE research on EduX-RAG (2025) reported that prompt-only cross-lingual chatbots exhibit a 27% language drift rate. When processing English source circulars, the LLM frequently responds in English even when the question was asked in Hindi or Tamil.

Our Technical Fix: Rather than hoping the LLM follows prompt instructions, our LangGraph architecture locks the user's language into the persistent conversation state. Node 5 inspects the final output: if an unexpected language change is detected, it automatically routes the text through an alignment pass, guaranteeing that response language strictly mirrors the student's input.

3. Clickable Source Citations and Complete Auditability (Paper 1 & 2 Finding)

The Vulnerability: Students cannot trust anonymous AI answers when dealing with strict academic deadlines and financial penalties.

Our Technical Fix: The generator outputs responses with structured reference tags. The ReactJS interface renders clickable citation cards displaying the circular title, department name, issue date, and page number. Clicking a citation opens the verified PDF directly to the relevant section.

4. Confidence Thresholding, Human Fallback, and SQL Logging

When similarity scores fall below 0.70 or documents are missing, the system refuses to guess. It presents the office contact details, visiting hours, and physical counter location. Simultaneously, it records the question, timestamp, and detected language into a SQL database. This creates an automated operational backlog for college staff, highlighting exactly which circulars or notices are missing from the public portal.

Technical Stack Matrix and Project Evaluation

Component selection justification and quantitative outcome evaluation plan

1. Comprehensive Technology Stack

System Layer	Technology Selected	Engineering Rationale
Frontend UI	ReactJS, Tailwind CSS	Responsive multilingual chat interface, citation cards, and audio/voice input support.
Backend API	Python, Flask	Lightweight, high-performance REST API routing requests between the UI and AI pipeline.
Orchestration	LangGraph, LangChain	Stateful graph workflow managing self-corrective loops, query rewriting, and language locking.
Web Scraper	BeautifulSoup (bs4)	Crawls live departmental notice boards for fast-breaking campus updates.
Dense Embedder	BGE-M3 (BAAI)	State-of-the-art multilingual dense vector model; proven 97.6% accuracy in published research.
Sparse Search	BM25 (Rank-BM25)	Guarantees exact matching for circular IDs, form numbers, and numerical dates.
Vector Store	FAISS / ChromaDB	In-memory and persistent vector index delivering sub-10 ms query retrieval latency.
Relational DB	SQL (SQLite / PostgreSQL)	Logs session history, audit trails, user feedback, and unresolved query escalation queues.

2. Project Evaluation Plan (Addressing the Research Gap in Paper 1)

Paper 1 noted that virtually all existing educational chatbot studies fail to measure real-world operational outcomes. We establish three measurable evaluation criteria for our project deployment:

- Grounded Accuracy & Faithfulness (RAGAS Metric): Using the RAGAS framework, we measure context relevance, answer faithfulness, and citation precision across an evaluation benchmark of 200 verified institutional questions.
- Cross-Lingual Consistency: Testing query resolution across Hindi, English, Marathi, and Hinglish to guarantee zero unintended language drift.
- Administrative Workload Deflection: Tracking the percentage of routine repetitive queries successfully resolved by the bot versus those escalated to office staff, targeting an 80%+ deflection rate.